

Claude CCAF - Actual Exam Questions

With researched answers and full explanations.

How to read this document. The source PDF contains questions only - it ships no answer key and no explanations. Every answer here was determined by research and is labelled with its basis:

- **source-key** - the same question appears in a source document that carries its own answer key, so the answer is taken from that key.
- **docs** - settled against Anthropic / Claude Code documentation.
- **principle** - transferred from a source-keyed question that tests the same rule.
- **reasoned** - argued from the options, with no direct citation.

Each answer also carries a confidence marker. 37 of 43 are high confidence; 6 are marked medium and are worth double-checking against your own materials before relying on them.

43 questions - generated 2026-08-24

Question 1

Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews-checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copy-pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly. What's the most effective approach?

- (A) Use the Bash tool with `curl` to call Jira's REST API, including authentication headers and parsing JSON responses inline.
- (B) Export Jira tickets to markdown files in the repository that the agent accesses using the Read tool.
- (C) Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.
- (D) Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code review workflow.

Answer: (C) Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.

Confidence: high | basis: principle - RAW-CG1-027 (build-vs-use): community servers first for standard integrations, custom only where no equivalent exists

EXPLANATION

Jira is a standard integration with well-maintained community MCP servers, and an MCP server exposes tickets and comments as discoverable tools the agent can use natively.

Why the others are wrong

- **A.** Bash plus `curl` bypasses the tool interface entirely: the agent has to hand-roll auth headers and parse JSON inline every time, with no structured tool description and no standardised error handling.
- **B.** An export goes stale the moment a ticket is updated, and the team explicitly wants live ticket data rather than a snapshot.
- **D.** Building custom is the right call only for a workflow with no community equivalent. Here the need is ordinary ticket data, so a custom build is unjustified maintenance.

Question 2

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module). What exploration strategy will most reliably identify all callers?

- (A) Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.
- (B) Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.
- (C) Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- (D) Use Grep to search for the function's original name across the codebase.

Answer: (B) Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.

Confidence: high | basis: reasoned - only option that accounts for the rename

EXPLANATION

The rename is the whole difficulty: `calculateTax` is re-exported as `computeOrderTax`, so no single search term covers every call site. Reading the library and wrapper modules first enumerates every name the function is exposed under, and Grep then finds callers of each.

Why the others are wrong

- **A.** Documentation describes intended usage, not actual call sites, and is routinely out of date - it cannot establish that removal is safe.
- **C.** Importing a module does not mean calling this function, and this misses files that receive the function indirectly. It also inverts the cost: reading every importing file is far more expensive than grepping known names.
- **D.** Searching only the original name misses every caller that uses a wrapper alias - exactly the callers the engineer is at risk of breaking.

Question 3

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

- (A) Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.
- (B) Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.
- (C) Use Glob to find files in directories commonly associated with error handling (such as errors/, exceptions/, or handlers/) across all services, then Read each matching file.
- (D) Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service.

Answer: (A) Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.

Confidence: high | basis: reasoned - content search on a distinctive literal

EXPLANATION

The error string is a distinctive literal that must appear in the source that emits it. Grep is the content-search tool, so searching SYNC_CONFLICT jumps straight to the emitting service; reading the matches then supplies context.

Why the others are wrong

- **B.** Importing the error module does not identify which service emits this particular message, and every service probably imports it.
- **C.** Glob matches paths, not contents, and assumes a directory convention the 12 services may not share.
- **D.** Reading architecture docs and then sweeping source files is the brute-force path - enormous context cost to find a string Grep locates in one call.

Question 4

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- (A) The agent automatically selects the most relevant server based on the request and loads only that server's tools.
- (B) Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.
- (C) You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- (D) The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.

Answer: (B) Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.

Confidence: high | basis: docs - Claude Code MCP reference: capability discovery (tools/list, prompts/list, resources/list) runs after each server connects; also stated verbatim in RAW-CG1-009's explanation

EXPLANATION

Claude Code runs capability discovery against every configured server as it connects, and the resulting tools are merged into one list the agent sees. The source guide puts it directly: MCP server boundaries are invisible to the agent - all tools from all connected servers appear in a single list. So the agent can call the git, Jira and docs tools in one task without any switching.

Why the others are wrong

- **A.** There is no per-request server selection step; nothing loads one server's tools to the exclusion of others.
- **C.** No per-turn server designation exists, and the agent is not limited to one server at a time - this request needs two servers in a single task.
- **D.** Discovery happens once at connection time, not by sequentially interrogating servers per call. Name prefixes disambiguate tools; they are not a routing mechanism the agent must drive.

Question 5

You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like `eval()` across a large codebase, which tool should your agent use for content search?

- (A) Use Grep to search for the pattern "eval(" across all files in the codebase.
- (B) Use Glob with a pattern like `/*eval*` to find files, then Read each matching file.
- (C) Read the project's main entry file and follow import statements to trace where `eval` might be used.
- (D) Use Bash to run `ls -R | grep eval` to recursively list files containing `eval`.

Answer: (A) Use Grep to search for the pattern "eval(" across all files in the codebase.

Confidence: high | basis: reasoned - Grep is the content-search tool

EXPLANATION

Finding occurrences of `eval()` is a content search across files, which is exactly Grep's job - one pass over the codebase returning the matching files and lines.

Why the others are wrong

- **B.** Glob matches file paths, not file contents. Files containing `eval()` are not named after it, so this finds the wrong set and then pays to read each one.
- **C.** Following imports from the entry point traces reachable modules, not textual occurrences, and misses dynamically referenced or unimported code.
- **D.** `ls -R | grep eval` filters the **file listing**, not file contents - it matches paths that contain "eval" and never opens a file.

Question 6

An engineer asks the agent to find all files in the monorepo that import the `@company/auth` package to understand how authentication is used across services. Which built-in tool is most appropriate for this task?

- (A) Glob, to find files with "auth" in their filename or path
- (B) Bash, to execute `find . -type d -name "*auth*"` and explore matching directories
- (C) Read, starting with `package.json` files to trace dependency declarations
- (D) Grep, to search for the import statement pattern across file contents

Answer: (D) Grep, to search for the import statement pattern across file contents

Confidence: high | basis: reasoned - content search, not filename search

EXPLANATION

An import statement is text inside files, so this is a content search: Grep for the import pattern for `@company/auth` returns exactly the files that import it.

Why the others are wrong

- **A.** Glob matches filenames and paths. A file that imports the `auth` package is usually not named "auth", so this both misses real importers and returns irrelevant files.
- **B.** `find . -type d` lists **directories** whose names match - it never inspects file contents, so it cannot answer the question asked.
- **C.** `package.json` records that a workspace depends on the package, not which source files import it, and in a monorepo the dependency is often hoisted.

Question 7

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

- (A) Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.
- (B) Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.
- (C) Create summaries of all source files before exploration begins, loading only these compressed representations into context.
- (D) Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.

Answer: (A) Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.

Confidence: high | basis: principle - RAW-CG1-043 and RAW-CG1-040 (scratchpad files persist findings across context boundaries)

EXPLANATION

This is context degradation: as exploration output accumulates, earlier findings get buried and the agent's answers drift toward generic patterns. A scratchpad file persists the essential findings on disk, decoupling what the agent knows from what currently fits in context - it can re-read the compact record instead of relying on a long transcript.

Why the others are wrong

- **B.** A larger context window postpones the problem without solving it; degradation is an attention-quality issue that appears well before the window is full.

- **C.** Pre-summarising every source file is enormous upfront work, and compressing before you know what matters discards the specifics (class names, signatures) the engineers need.
- **D.** Clearing on a timer throws away valid findings on a schedule unrelated to the work, forcing exactly the re-explaining the engineers are complaining about.

Question 8

Your productivity agent connects to three MCP servers: an issue tracker (search_issues, get_issue, create_comment), a documentation wiki (search_docs, get_page, list_spaces), and a database explorer (run_query, get_schema, list_databases). When engineers ask cross-system questions like "What database tables are affected by the authentication refactor in PROJ-1234?", monitoring shows the agent makes 8-10 sequential tool calls, frequently issues exploratory calls because it lacks visibility into what content each server contains, and exhausts context space before completing complex investigations. What architectural change best leverages MCP capabilities to address these issues?

- (A) Add a prepare_investigation tool to each server that accepts a natural language question and returns relevant content summaries
- (B) Add an orchestrator that routes questions to a single server based on keywords
- (C) Consolidate all three servers into a unified MCP server with cross-referencing capabilities
- (D) Expose each server's content catalog as MCP resources-issue summaries, documentation hierarchy, database schemas

Answer: (D) Expose each server's content catalog as MCP resources-issue summaries, documentation hierarchy, database schemas

Confidence: high | basis: source-key - RAW-CG1-038 answers this pattern with MCP resources; corroborated by the MCP resources primitive in the Claude Code docs

EXPLANATION

The root problem is that the agent has no visibility into what each server holds, so it burns turns on exploratory calls. MCP resources exist for exactly this: they expose a content catalogue - issue summaries, documentation hierarchy, database schemas - so the agent can see what is available upfront and go straight to the right call instead of discovering by trial.

Why the others are wrong

- **A.** A prepare_investigation tool keeps an exploratory call on every task, which is the very overhead to be removed, and pushes retrieval logic into each server.
- **B.** Keyword routing to a single server breaks genuinely cross-system questions - this question spans an issue tracker, a wiki and a database at once.
- **C.** Consolidating three servers is a large refactor that does not address visibility; the merged server still exposes tools without a catalogue.

Question 9

After adding an MCP server with specialized code refactoring tools (extract_function, rename_variable, inline_function), you notice the agent still uses basic text manipulation via Write and Bash sed commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like "extract_function: Extracts a function from code." What's the most effective way to improve adoption of the MCP refactoring tools?

- (A) Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.
- (B) Remove the Write tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.
- (C) Accept this as expected behavior since simpler tools like sed are more predictable than specialized refactoring tools.
- (D) Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.

Answer: (A) Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.

Confidence: high | basis: source-key - RAW-CG1-001 is the same scenario and answers 'enhance the sparse description'

EXPLANATION

When an MCP tool's description is sparse, the agent falls back to familiar general-purpose tools like Write and Bash. The description is the primary signal the model uses to choose a tool, so stating when each refactoring tool is preferable to text manipulation - and what it takes and returns - fixes the selection at its source.

Why the others are wrong

- **B.** Removing Write is a blunt instrument that breaks legitimate non-refactoring work, and it forces the tools rather than making them the better choice.
- **C.** Accepting it forfeits the AST-level correctness the refactoring tools provide; sed is predictable but not safe for renaming symbols.
- **D.** An external classifier that pre-routes requests is significant infrastructure layered over a one-line configuration fix, and it removes the agent's ability to reason about tool choice.

Question 10

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What's the most effective approach?

- (A) Resume the subagent from its previous transcript without mentioning the changes-the architecture understanding remains valid.
- (B) Resume the subagent from its previous transcript and inform it about the renamed functions.
- (C) Launch a fresh subagent with a summary of prior findings.
- (D) Launch a fresh subagent and include the prior transcript in the initial prompt for context.

Answer: (C) Launch a fresh subagent with a summary of prior findings.

Confidence: medium | basis: principle - RAW-CG1-012's explanation states the rule: a fresh start with summary injection is best when tool results are stale

EXPLANATION

The transcript's value is the documented data flows, not the 47 raw file reads - and those reads are now stale, because a merged PR renamed two functions the transcript still refers to by their old names. The guide's rule for exactly this situation is that a fresh start with summary injection is right when tool results have gone stale: the fresh subagent carries forward the durable findings and re-reads the current code.

Why the others are wrong

- **A.** Resuming while ignoring the renames leaves the agent reasoning from stale identifiers, so it will reference functions that no longer exist.
- **B.** Telling it about the renames patches the two functions you know about, but the resumed transcript still carries 47 files of raw, now-outdated content - expensive and still stale in ways nobody has audited.
- **D.** Pasting the whole prior transcript into a fresh prompt reproduces the staleness and the context bloat, discarding the benefit of starting fresh.

Question 11

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

- (A) Use Glob to find files matching common caching patterns (`*cache*.py`, `caching/`), prioritize the largest files by reading them first, then check smaller files for gaps.
- (B) Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.
- (C) Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- (D) Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

Answer: (D) Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

Confidence: medium | basis: reasoned - structure-first traversal under an explicit context constraint

EXPLANATION

Roughly 8,000 lines cannot all be held in context, so the question is which reading order buys the most understanding per token. Following imports and class hierarchies to the base cache class gives the interface that every decorator, middleware and service implements; from there, only the specific invalidation implementations need reading. Structure first, then targeted depth.

Why the others are wrong

- **A.** Prioritising the largest files is an arbitrary proxy for importance, and Glob patterns re-discover files that Grep has already located.
- **B.** Reading isolated line ranges around "invalidate" and "expire" yields fragments without the surrounding class and interface context needed to add a new trigger correctly.
- **C.** Sequentially loading all 15 files is precisely what the context constraint forbids - it fills the window with implementation detail before any structure is established.

Question 12

Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR. Which task decomposition pattern is most appropriate for this workflow?

- (A) Routing-classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.
- (B) Prompt chaining-break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.
- (C) Orchestrator-workers-have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.
- (D) Single comprehensive prompt-include all instructions in one prompt and let the model handle all three aspects simultaneously.

Answer: (B) Prompt chaining-break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.

Confidence: high | basis: docs - Anthropic's 'Building effective agents': prompt chaining fits fixed, decomposable subtasks; orchestrator-workers fits dynamically determined ones

EXPLANATION

The decisive detail is that the workflow is identical for every PR - the same three aspects, in the same order, every time. Prompt chaining is the pattern for a task that decomposes into a fixed sequence of subtasks, and giving each aspect its own step also avoids the attention dilution that degrades a single combined pass.

Why the others are wrong

- **A.** Routing classifies an input and sends it down **different** paths. Here every PR needs all three checks, so there is nothing to route.
- **C.** Orchestrator-workers is for work whose subtasks cannot be known in advance. These subtasks are fixed, so the dynamic planning step adds cost and variance for nothing.
- **D.** One comprehensive prompt is what causes uneven quality across the three aspects; attention spreads thin and later aspects are handled superficially.

Question 13

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its `old_string` parameter cannot find unique text to match - the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

- (A) Use Read to load the file, add the function at the appropriate location, then Write the updated file
- (B) Use Edit's `replace_all` parameter to target a common pattern and embed the new function in the replacement text
- (C) Use Edit with an extremely long `old_string` capturing 30+ lines of context to guarantee uniqueness
- (D) Use Bash to append the function definition to the end of the file using heredoc syntax

Answer: (A) Use Read to load the file, add the function at the appropriate location, then Write the updated file

Confidence: high | basis: source-key - matched to a keyed question in the built-in tools chapter

EXPLANATION

Edit needs a unique `old_string`, and this file cannot supply one. Read plus Write sidesteps the uniqueness requirement entirely: load the current contents, place the function where it belongs, and write the file back whole.

Why the others are wrong

- **B.** `replace_all` deliberately edits **every** occurrence of a repeated pattern - in a file full of repetitive structure that corrupts many sites and duplicates the new function.
- **C.** A 30-line `old_string` is brittle: it must match whitespace and content exactly, it is easy to get subtly wrong, and it fails again the moment the surrounding code shifts.
- **D.** Appending to the end of the file puts the helper in the wrong place - the requirement is to insert it between two specific functions.

Question 14

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services. What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- (A) Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.
- (B) Read all files containing "auth", "login", "permission", or "token" in their content or filename.
- (C) Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.

(D) Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.

Answer: (A) Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.

Confidence: medium | basis: principle - RAW-CG1-040 (delegate exploration to subagents to isolate verbose output) and RAW-CG1-008 (parallelise independent per-service work)

EXPLANATION

The constraint stated in the question is context limits across 800+ files and multiple services, and the goal is an architectural overview. Subagents explore in their own context windows, so the verbose file reading never lands in the main conversation; running one per service exploits the fact that the services are independent, and the coordinator synthesises the returned findings into the overview the engineer asked for.

Why the others are wrong

- **B.** Reading every file that mentions "auth", "login", "permission" or "token" across 800+ files is unbounded and would exhaust the context window without producing a structured picture.
- **C.** Asking a brand-new engineer to name the 10-15 most important files inverts the request - identifying those files is the very thing they need help with.
- **D.** Incremental Grep-and-follow is a sound technique, but performed serially in one context across multiple services it accumulates exactly the degradation the question flags, and it traces one flow rather than mapping the architecture.

Question 15

Your agent has spent 25 minutes exploring a game engine's rendering subsystem-reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier. What's the most effective approach?

- (A) Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.
- (B) Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- (C) Continue in the current context with more targeted prompts referencing the specific classes by name.
- (D) Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.

Answer: (D) Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.

Confidence: high | basis: principle - RAW-CG1-033 (consolidate key findings into a structured block) plus subagent context isolation

EXPLANATION

The symptom - generic "typical rendering patterns" instead of the VulkanPipeline and FrameGraph specifics it found earlier - is context degradation. The new question is about how physics **integrates with rendering**, so the rendering knowledge must survive. Summarising those findings consolidates them into a compact, recent block, and spawning a subagent for physics with that summary keeps the new exploration's verbose output out of the already-degraded main context.

Why the others are wrong

- **A.** `/clear` discards the rendering knowledge the integration question depends on, forcing a re-exploration that has already been paid for.
- **B.** Spawning a physics subagent without passing the rendering summary means it cannot reason about the integration, and leaves you to reconcile its findings against a degraded main context.
- **C.** Naming the classes in prompts treats the symptom. The degradation is structural - continuing in the same saturated context will keep losing specifics.

Question 16

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions ($\geq 85\%$) also contain errors-cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time. What approach is most effective?

- (A) Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.
- (B) Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.
- (C) Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.
- (D) Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.

Answer: (D) Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.

Confidence: high | basis: principle - RAW-CG1-039 (stratified sampling allocates review effort and supports measurement)

EXPLANATION

Two requirements are stated: catch errors that confidence scores do not predict, and **measure whether the error rate improves over time**. Only a fixed-percentage sampling regime gives an unbiased, repeatable estimate you can track across releases, and reviewing a random cross-section also surfaces novel error patterns rather than the ones you already anticipated.

Why the others are wrong

- **A.** Re-extracting from the same document tends to reproduce the same plausible-but-wrong value, so agreement is not evidence of correctness - self-consistency is not accuracy.
- **B.** Dropping the threshold to 70% floods reviewers with the low-confidence band that was already being caught, without touching the high-confidence errors that are the actual problem, and yields no measurement.
- **C.** Heuristics for comparison tables and appendices only catch the error sources already identified; they cannot detect new ones and provide no error-rate metric.

Question 17

After deployment, you find that 12% of extractions contain semantic errors that pass JSON schema validation (e.g., a duration like "30 minutes" incorrectly placed in an ingredient quantity field). Human reviewers have capacity to check only 20% of extractions. Which approach most effectively allocates reviewer attention?

- (A) Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types.
- (B) Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.
- (C) Prioritize review of all extractions where required fields are empty or explicitly marked as not found.
- (D) Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set.

Answer: (B) Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.

Confidence: medium | basis: reasoned - unbiased measurement plus pattern discovery under a fixed review budget

EXPLANATION

The errors are semantic and pass schema validation, so there is no structural signal that reliably flags them, and no risk stratification is given. Spending the fixed 20% budget on a random sample gives an unbiased accuracy estimate and, because corrections are recorded, surfaces the recurring error patterns that drive the next round of fixes.

Why the others are wrong

- **A.** Formatting anomalies are a proxy for difficulty, not for the semantic mix-ups described, and concentrating review there yields a biased estimate that says nothing about the rest of the corpus.
- **C.** Empty and not-found fields are the visible failures. The errors here are **populated** fields holding the wrong value, which this filter never looks at.
- **D.** Field-level self-reported confidence is poorly calibrated, and the calibration this depends on needs a labelled validation set that does not exist yet - random sampling is how you would build one.

Question 18

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence $\geq 90\%$ have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- (A) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- (B) Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.
- (C) Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.
- (D) Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.

Answer: (A) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-038), answer confirmed

EXPLANATION

97% is an aggregate, and aggregates hide segments. If high confidence is well calibrated on invoices but poorly calibrated on one document type or one field, automating the whole band ships a systematic error into production while the overall number still looks healthy. Breaking accuracy down by document type and field is what establishes that the confidence signal is trustworthy everywhere before you rely on it.

Why the others are wrong

- **B.** Threshold tuning optimises a cutoff on a signal whose reliability has not yet been established - the right sequence is validate first, tune second.
- **C.** Checking that 97% satisfies downstream consumers is a worthwhile requirements question, but it accepts the aggregate at face value rather than testing whether it holds per segment.
- **D.** A pilot is useful, but deploying to 25% of live traffic before the segment analysis is exactly the avoidable risk - the analysis can be done offline on data you already have.

Question 19

Your extraction uses tool use with a JSON schema where `property_type` is defined as an enum: `['house', 'apartment', 'condo', 'townhouse']`. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types-"studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse"-and new types continue appearing regularly. What's the most effective long-term solution?

- (A) Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.
- (B) Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.
- (C) Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.
- (D) Add an "other" value to your enum with a separate `property_type_detail` string field for specifics when "other" is selected.

Answer: (D) Add an "other" value to your enum with a separate `property_type_detail` string field for specifics when "other" is selected.

Confidence: high | basis: principle - RAW-CG1-003: an enum plus an 'other' value handles edge cases without allowing free-text drift

EXPLANATION

The open-ended tail is the real constraint: new property types keep appearing, so no fixed list stays complete. An other enum value with a companion `property_type_detail` string keeps the common cases constrained and analysable while giving the model a legitimate place to put the unusual ones - so validation stops failing and no information is lost.

Why the others are wrong

- **A.** A free-form string reintroduces exactly the drift enums prevent - "studio", "Studio", "studio apartment" - and pushes an endless normalisation-maintenance burden downstream.
- **B.** Continuously expanding the enum is a permanent chase; the question states new types keep arriving, so the list is out of date between every release.
- **C.** Forcing unusual types onto the nearest existing value silently records incorrect data - a converted warehouse is not a house.

Question 20

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field-sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source. What's the most effective way to improve extraction consistency?

- (A) Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats
- (B) Set temperature to 0 to eliminate randomness and ensure deterministic outputs
- (C) Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability
- (D) Make the "materials" field required instead of optional in the schema to force the model to always extract a value

Answer: (A) Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats

Confidence: high | basis: principle - RAW-CG1-018 and RAW-CG1-051: few-shot examples are the intervention when a well-specified schema still yields inconsistent formatting

EXPLANATION

The schema already defines the field, so the gap is not structural - it is that "materials" has no agreed output convention, and omission shows the model is unsure when to populate it. Two or three complete input-output pairs demonstrate the intended granularity and format and show the field being filled from a source that mentions materials in passing, which is what teaches consistency.

Why the others are wrong

- **B.** Temperature 0 removes sampling variance, not ambiguity. With no defined convention the model will now be consistently arbitrary, and it does not explain the omissions.
- **C.** Inconsistent formatting of an underspecified field is not a capability ceiling; a stronger model still has to guess which convention you want.
- **D.** Making the field required forces a value even when none is present, converting omissions into fabrications - and it does nothing about the formatting inconsistency.

Question 21

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned." What's the most effective improvement?

- (A) Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.
- (B) Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- (C) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- (D) Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

Answer: (C) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-036), answer confirmed

EXPLANATION

All three symptoms are judgement calls the prompt never makes explicit: where a compound phrase splits, what counts as "mentioned", and how granular an entry should be. Few-shot examples demonstrate all three at once - the model sees "Python and SQL" become two entries, sees an implied skill left out, and sees the expected entry count - which teaches the criteria rather than asserting them.

Why the others are wrong

- **A.** Confidence and source-quote metadata document the inconsistency without resolving it; the model still has to decide how to split and what to include.
- **B.** "10-20 maximum" is an arbitrary cap that truncates genuinely skill-dense documents, and it still does not say how to handle compound phrases or implied skills.
- **D.** Canonical-taxonomy normalisation is downstream cleanup. It cannot recover a skill that was never extracted or remove one that was inferred but never stated.

Question 22

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats - inline citations vs. bibliographies, methodology sections vs. details embedded in introductions. What's the most effective way to address these failures?

- (A) Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.
- (B) Add few-shot examples demonstrating extractions from documents with varied structures - showing how to identify citations in different formats and locate methodology details across section types.
- (C) Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.
- (D) Implement retry logic that re-sends requests when validation detects empty required fields.

Answer: (B) Add few-shot examples demonstrating extractions from documents with varied structures - showing how to identify citations in different formats and locate methodology details across section types.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-029), answer confirmed

EXPLANATION

Schema enforcement guarantees shape, not recall - the JSON is valid, but the fields are empty because the model does not recognise citations rendered as a bibliography or methodology buried in an introduction. Few-shot examples drawn from documents with those varied structures teach the model to find the information wherever it lives, which is the actual failure.

Why the others are wrong

- **A.** Regex over citation patterns and methodology keywords is brittle across the varied formats described, and methodology prose in particular has no reliable lexical signature.
- **C.** Making the fields optional lowers the standard rather than fixing extraction, and pushes the work onto manual review.
- **D.** Retrying an unchanged request reproduces the same miss - nothing about the second attempt helps the model recognise an unfamiliar citation format.

Question 23

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article-for example, outputting "500" for attendee_count when the source contains no attendance information. What's the most effective way to reduce these false extractions?

- (A) Add prompt instructions to return null for any field where information is not directly stated in the source.
- (B) Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.
- (C) Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.
- (D) Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.

Answer: (A) Add prompt instructions to return null for any field where information is not directly stated in the source.

Confidence: medium | basis: principle - RAW-CG1-015: fabrication is driven by the absence of a legitimate 'not applicable' path; here the schema already provides one, so the instruction is what is missing

EXPLANATION

The schema is already fully nullable, so the model has a legitimate way to say "not present" - it simply is not being told to use it. An explicit instruction to return null for anything not directly stated in the source closes the gap between the schema's permission and the model's behaviour, at no extra inference cost.

Why the others are wrong

- **B.** Making every field required removes the null escape hatch and **guarantees** fabrication - the model must emit a value even when the article contains none. This is the opposite of the fix.
- **C.** A second verification call catches some fabrications after the fact, but doubles cost and latency to patch a problem the first call can be told not to create.
- **D.** A stronger model reduces the tendency without addressing the cause; the instruction is missing regardless of tier.

Question 24

Your extraction pipeline occasionally receives responses that don't parse as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it. What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

- (A) Implement a retry loop that catches JSON parse errors and re-prompts Claude with the error details, asking it to correct the malformed output.
- (B) Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.
- (C) Define a tool with a JSON schema specifying the expected structure, using tool_use to constrain Claude's output to schema-compliant JSON.
- (D) Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.

Answer: (C) Define a tool with a JSON schema specifying the expected structure, using tool_use to constrain Claude's output to schema-compliant JSON.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-035); corroborated by Anthropic's tool-use documentation on schema-constrained output

EXPLANATION

Parsing JSON out of free text is fundamentally unreliable because nothing constrains what the model emits. Defining a tool whose input schema is your target structure moves the guarantee to generation time - the model fills the schema, so the output is valid and schema-compliant by construction rather than by inspection.

Why the others are wrong

- **A.** A retry loop is reactive: it costs an extra call on every failure and still offers no guarantee the corrected output parses.
- **B.** Regex extraction is a workaround for the symptom. It cannot fix genuinely malformed JSON and breaks on nested structures and stray braces in string values.
- **D.** Formatting instructions are probabilistic guidance. They reduce the failure rate but cannot eliminate it, which is why the pipeline is failing today.

Question 25

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- (A) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.
- (B) Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- (C) Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- (D) Submit all documents to the Batch API with custom_ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

Answer: (A) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-041); Batch API's asynchronous 50% discount is documented

EXPLANATION

The two document types have genuinely different requirements, so they should use different APIs. Standard monthly reports are archived and latency-tolerant, which is exactly the Batch API's trade - asynchronous processing at roughly half the cost. Urgent exception reports carry a hard 30-minute alerting deadline, so they belong on the real-time Messages API.

Why the others are wrong

- **B.** Sending everything real-time meets the deadline but forfeits the batch discount on the bulk of the volume, which is the stated cost goal.
- **C.** Hourly batching adds queueing delay on top of batch processing time and cannot guarantee a 30-minute alert.
- **D.** The Batch API has no latency SLA, so urgent reports may return well past 30 minutes - and "delayed alerts" concedes the requirement is missed.

Question 45

You are integrating Claude Code into your CI/CD pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. Your automated review generates many findings per PR, but developer feedback shows roughly half are dismissed as "not worth addressing." Analysis reveals dismissed findings are often technically accurate but involve minor style preferences or patterns acceptable in your codebase. Before adding infrastructure complexity, what prompt design change would most effectively reduce dismissals while maintaining detection of genuine issues?

- (A) Append instructions to "only report findings you are highly confident are genuine problems"
- (B) Implement a secondary classification model that filters Claude's findings based on predicted developer acceptance
- (C) Add explicit criteria defining which issues to report (bugs, security) versus skip (minor style, local patterns)
- (D) Add instructions asking Claude to rate each finding's confidence (1-10) and only include findings rated 8 or higher

Answer: (C) Add explicit criteria defining which issues to report (bugs, security) versus skip (minor style, local patterns)

Confidence: high | basis: principle - RAW-CG1-055 and RAW-CG1-048: replace vague hedging with explicit categorical criteria

EXPLANATION

The dismissed findings are technically correct but out of scope for what this team wants flagged, so the problem is that scope was never defined. Explicit criteria - report bugs and security issues, skip minor style and accepted local patterns - give the model the decision boundary it is currently guessing at, and they do it in the prompt, before any infrastructure is added.

Why the others are wrong

- **A.** "Only report findings you are highly confident about" is exactly the vague hedge that fails. The dismissed findings **are** genuine, so confidence is not the axis that separates them.
- **B.** A secondary classifier is the infrastructure complexity the question asks you to avoid until prompt design has been tried.
- **D.** Self-rated 1-10 confidence is poorly calibrated and, again, filters on the wrong dimension: a correct-but-unwanted style nit can score a confident 9.

Question 46

You are building a multi-agent research system using the Claude Agent SDK. Production monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent-neither requires the other's output. How should you modify the system to run these subagents concurrently?

- (A) Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator-subagent pair, then aggregates results.
- (B) Switch both subagents to use a Haiku-tier model instead of Sonnet to reduce their individual execution time.
- (C) Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.
- (D) Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time.

Answer: (C) Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-007); corroborated by the documented parallel tool-use contract

EXPLANATION

Parallelism here is structural, not behavioural. A single assistant turn may contain multiple `tool_use` blocks, and the harness executes the calls that arrive together concurrently. So emitting both Task calls in one response message is what makes the two subagents run at the same time; total latency becomes the slower of the two rather than their sum.

Why the others are wrong

- **A.** An external async layer that duplicates coordinator-subagent pairs invents infrastructure to solve a problem the tool-calling loop already handles natively.
- **B.** A faster model tier shortens each call but leaves them sequential - it optimises per-call speed, not concurrency.
- **D.** Asking the coordinator in the system prompt to "be parallel" is prompt-layer guidance with no structural guarantee; it may still emit the calls across separate turns.

Question 47

You are building a multi-agent research system using the Claude Agent SDK. In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

- (A) Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.
- (B) Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.
- (C) Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.
- (D) Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

Answer: (B) Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

Confidence: high | basis: source-key - RAW-CG1-046 answers this pattern: the coordinator should select which subagents each query needs

EXPLANATION

Deciding which subagents a query needs is precisely the coordinator's job, and a model-driven decision adapts as the query distribution evolves - which the question says it is doing. A simple factual lookup gets routed to web search alone; a comparative study still gets the full pipeline, with no rule table to maintain.

Why the others are wrong

- **A.** A hard fast-path/full-pipeline split is a two-bucket approximation of a diverse distribution: anything mid-complexity is either under-served or over-served.
- **C.** Pattern-based routing on query structure needs its categories maintained by hand and misclassifies phrasings the patterns never anticipated - brittle against an evolving distribution.
- **D.** Training and periodically retraining a classifier is heavy machinery requiring labelled history, to reproduce a judgement the coordinator can already make.

Question 48

You are building a multi-agent research system using the Claude Agent SDK. You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization-the synthesis agent attempts web searches, and the report generator tries to analyze documents. What is the primary cause of this poor tool selection behavior?

- (A) The agents' role descriptions in their system prompts conflict with having access to tools outside that role.
- (B) The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.
- (C) Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.
- (D) The tool definitions consume too much context window space, leaving insufficient room for task content.

Answer: (C) Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.

Confidence: high | basis: principle - RAW-CG1-009 and RAW-CG1-029: tool selection degrades beyond roughly 4-5 tools per agent

EXPLANATION

Tool-selection reliability falls off as the candidate set grows; the practical guidance is roughly 4-5 tools per agent. With all 18 tools exposed to every subagent, each one is choosing from a set far beyond that threshold, so it sometimes picks a tool belonging to another specialisation. The fix is scoped tool access - give each subagent only the tools its role needs.

Why the others are wrong

- **A.** Role descriptions and tool access are not in conflict; the prompt describes the role correctly. Prompt wording is probabilistic guidance that cannot deterministically prevent a wrong pick.
- **B.** Coordinator tracking is not the issue - the misfire happens inside a subagent that has been handed tools it should never have had.
- **D.** 18 tool definitions do consume context, but the symptom is **wrong tool chosen**, not **ran out of room**. Context pressure would degrade the task content, not bias selection toward off-role tools.

Question 49

You are building a multi-agent research system using the Claude Agent SDK. The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

- (A) Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.
- (B) Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.
- (C) Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.
- (D) Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."

Answer: (A) Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.

Confidence: high | basis: principle - RAW-CG1-007: replace hardcoded procedure with model-driven decisions when the situation demands nuance

EXPLANATION

All three symptoms trace to the same cause: the coordinator dictates exact procedure, so the subagent has no latitude to adapt when reality does not match the plan. Specifying goals and quality criteria instead - coverage breadth, source diversity, recency - tells the subagent what a good result looks like and lets it choose the strategy, which is what lets it recover from failed searches, handle emerging topics, and follow up tangential leads.

Why the others are wrong

- **B.** A well-defined/exploratory classifier adds a step that still hands down procedural instructions in the well-defined branch, so the same rigidity resurfaces there.
- **C.** Stripping all guidance to "research X thoroughly" overcorrects - the subagent loses the quality bar entirely and results become unpredictable.
- **D.** Fallback directives patch the one failure mode you anticipated (too few results) and leave the other two untouched; it is still a procedure, just a longer one.

Question 50

You are building a multi-agent research system using the Claude Agent SDK. When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- (A) Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents
- (B) Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis
- (C) Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations
- (D) Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity

Answer: (B) Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-006), answer confirmed

EXPLANATION

The precedents are independent, so they can be analysed concurrently. Having the coordinator spawn parallel subagents over subsets and aggregate their results cuts latency while keeping orchestration centralised - the coordinator still sees every delegation and every result, which is what preserves monitoring and debugging.

Why the others are wrong

- **A.** A worker-pool message queue improves throughput but adds infrastructure and moves execution out of the coordinator's view, reducing debuggability.
- **C.** Letting the subagent spawn its own subagents dynamically decentralises orchestration; the coordinator loses visibility into agents it never created.
- **D.** A recursive hierarchy compounds that problem - execution paths become deep and hard to trace, for no gain over one level of fan-out.

Question 55 (Set A)

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

- (A) Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results
- (B) Include the complete findings from both subagents directly in the synthesis subagent's prompt
- (C) Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator
- (D) Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks

Answer: (A) Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results

Confidence: medium | basis: source-key - matched to a keyed question (RAW-EH-012), whose source answer is the shared-memory-store option; note this trades against the 'pass findings explicitly' rule elsewhere in the syllabus

EXPLANATION

At production scale the findings from two subagents can be large, so inlining everything into the synthesis prompt risks exhausting context. Passing reference identifiers with read access to a shared store preserves full fidelity while keeping the prompt small - the synthesis agent retrieves exactly what it needs.

Why the others are wrong

- **B.** Including the complete findings inline is the simple approach and is correct at small scale, but it does not scale: large outputs from both subagents can exceed the context limit.
- **C.** There is no automatic context inheritance between agents - a brief task description alone leaves the synthesis agent with nothing to synthesise.
- **D.** Callback tools let subagents pull from each other, which couples them together and breaks the coordinator's role as the single point of data flow.

Note: a closely related question elsewhere in this syllabus answers 'pass the complete findings in the prompt', on the grounds that subagents share no memory. Both appear in the source material; the distinguishing factor is scale - reach for the store when the findings are too large to inline.

Question 56 (Set A)

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

- (A) Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.
- (B) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.
- (C) Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.
- (D) Have each agent maintain its own persistent state file and reload it independently at the start of each session.

Answer: (B) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-004), answer confirmed

EXPLANATION

Structured exports give high information fidelity - each agent's completed work is preserved in a defined shape - while the coordinator loading a manifest and injecting only the relevant state keeps context efficient. The coordinator stays in control of what each agent receives on resume, so no work is repeated and no findings are blurred.

Why the others are wrong

- **A.** A vector store introduces probabilistic recall: semantic search may miss or distort critical state, which is unacceptable when the goal is fidelity during recovery.
- **C.** Conversation logs are verbose and unstructured, so replaying them bloats context without guaranteeing the specific state each agent needs.
- **D.** Independent per-agent state files decentralise control and drift out of sync, since no component ensures the agents resume from a consistent view.

Question 57 (Set A)

The document analysis agent has a single `analyze_document` tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

- (A) Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent
- (B) Keep the single tool but add an `analysis_type` enum parameter requiring explicit selection between extraction, summarization, and verification modes
- (C) Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats
- (D) Split the generic tool into purpose-specific tools-`extract_data_points`, `summarize_content`, `verify_claim_against_source`-each with defined input/output contracts

Answer: (D) Split the generic tool into purpose-specific tools-`extract_data_points`, `summarize_content`, `verify_claim_against_source`-each with defined input/output contracts

Confidence: high | basis: principle - RAW-CG2-019 and RAW-EH-023/026: split an overloaded generic tool into purpose-specific tools with defined contracts

EXPLANATION

One tool with a free-text instruction has no output contract, so the same phrasing can yield a narrative summary or a data table. Splitting it into `extract_data_points`, `summarize_content` and `verify_claim_against_source` makes the intent structural: each tool has a defined input and output shape, so the ambiguity that drives the 35% re-request rate cannot be expressed.

Why the others are wrong

- **A.** Pre-classification by the coordinator adds a step while still funnelling everything through the same ambiguous free-text parameter.
- **B.** An `analysis_type` enum is a real improvement and close to correct, but the single tool still returns one undifferentiated output shape - the modes share a return contract rather than each having their own.
- **C.** Description examples are advisory guidance; they make the right mapping likelier without making the wrong one impossible.

Question 58 (Set A)

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

- (A) Update the synthesis agent to render each content type appropriately-financial data as tables, news as prose
- (B) Standardize all subagent outputs to prose summaries with inline citations
- (C) Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation
- (D) Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence

Answer: (A) Update the synthesis agent to render each content type appropriately-financial data as tables, news as prose

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-009), answer confirmed

EXPLANATION

Each source has a natural shape, and flattening everything to bullets destroys it. Rendering each content type appropriately - financial figures as tables, news as prose - preserves the comparability of the numbers and the narrative flow of the reporting, which is exactly what the briefings lost.

Why the others are wrong

- **B.** Standardising on prose destroys tabular comparability for the financial data, trading one loss for another.
- **C.** A common intermediate representation is a conversion step that still has to be rendered at the end, and it tends to converge on generic formatting - the original problem.
- **D.** A uniform claim/evidence/source/confidence JSON improves structure for the pipeline but does not improve the human-readable briefing, which is what the question asks about.

Question 59 (Set A)

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

- (A) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.
- (B) Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.
- (C) Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.
- (D) Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

Answer: (A) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.

Confidence: high | basis: source-key - identical to a keyed question (RAW-EH-005), answer confirmed

EXPLANATION

The coordinator owns iterative refinement. Evaluating the synthesis for gaps and re-delegating targeted follow-up queries closes the specific holes before the report is written, without discarding the work already done - a feedback loop rather than a one-way pipeline.

Why the others are wrong

- **B.** Noting the unanswered questions improves transparency but leaves the report incomplete; the stated goal is completeness.
- **C.** Giving the synthesis agent its own web search tools breaks separation of concerns and bypasses the coordinator's control over delegation.
- **D.** Broadening the initial queries is a blunt, upfront guess that costs more on every run and still cannot guarantee it covers gaps only discovered after synthesis.

Question 60 (Set A)

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations-the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

- (A) Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.
- (B) Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.
- (C) Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).
- (D) Have the report generator query the web search agent to re-locate sources for claims in the final report.

Answer: (C) Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

Confidence: high | basis: principle - RAW-EH-011 and RAW-CG1-016: preserve attribution as structured claim-to-source data, not as prose

EXPLANATION

Attribution is lost because it lives in prose that gets rewritten at every summarisation step. Having each agent emit structured data that separates the content summary from source metadata - URLs, document names, page numbers - keeps the mapping in data fields that survive rewriting, so the report generator can still attribute each claim.

Why the others are wrong

- **A.** Passing full raw outputs may preserve the sources but is enormously inefficient and risks exceeding context limits; it also does not map specific claims to specific sources.
- **B.** Inline citations are prose, and prose is exactly what summarisation rewrites - the references get stripped or detached from their claims.
- **D.** Re-locating sources after the fact is post-hoc reconstruction, which is error-prone and can confidently misattribute a claim to the wrong source.

Question 55 (Set B)

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations-the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

- (A) Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).
- (B) Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.
- (C) Have the report generator query the web search agent to re-locate sources for claims in the final report.
- (D) Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.

Answer: (A) Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

Confidence: high | basis: same question as 60 (Set A) with options reordered - principle: RAW-EH-011 and RAW-CG1-016

EXPLANATION

Attribution is lost because it lives in prose that gets rewritten at every summarisation step. Having each agent emit structured data that separates the content summary from source metadata - URLs, document names, page numbers - keeps the mapping in data fields that survive rewriting, so the report generator can still attribute each claim.

Why the others are wrong

- **B.** Passing full raw outputs may preserve the sources but is enormously inefficient and risks exceeding context limits; it also does not map specific claims to specific sources.
- **C.** Re-locating sources after the fact is post-hoc reconstruction, which is error-prone and can confidently misattribute a claim to the wrong source.
- **D.** Inline citations are prose, and prose is exactly what summarisation rewrites - the references get stripped or detached from their claims.

Question 56 (Set B)

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

- (A) Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.
- (B) Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.
- (C) Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.
- (D) Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.

Answer: (B) Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

Confidence: high | basis: same question as 47 with options reordered - RAW-CG1-046

EXPLANATION

Deciding which subagents a query needs is precisely the coordinator's job, and a model-driven decision adapts as the query distribution evolves - which the question says it is doing. A simple factual lookup gets routed to web search alone; a comparative study still gets the full pipeline, with no rule table to maintain.

Why the others are wrong

- **A.** Training and periodically retraining a classifier is heavy machinery requiring labelled history, to reproduce a judgement the coordinator can already make.
- **C.** A hard fast-path/full-pipeline split is a two-bucket approximation of a diverse distribution: anything mid-complexity is either under-served or over-served.
- **D.** Pattern-based routing on query structure needs its categories maintained by hand and misclassifies phrasings the patterns never anticipated - brittle against an evolving distribution.

Question 57 (Set B)

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

- (A) Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."
- (B) Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.
- (C) Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.
- (D) Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.

Answer: (C) Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.

Confidence: high | basis: same question as 49 with options reordered - RAW-CG1-007

EXPLANATION

All three symptoms trace to the same cause: the coordinator dictates exact procedure, so the subagent has no latitude to adapt when reality does not match the plan. Specifying goals and quality criteria instead - coverage breadth, source diversity, recency - tells the subagent what a good result looks like and lets it choose the strategy, which is what lets it recover from failed searches, handle emerging topics, and follow up tangential leads.

Why the others are wrong

- **A.** Fallback directives patch the one failure mode you anticipated (too few results) and leave the other two untouched; it is still a procedure, just a longer one.
- **B.** A well-defined/exploratory classifier adds a step that still hands down procedural instructions in the well-defined branch, so the same rigidity resurfaces there.
- **D.** Stripping all guidance to "research X thoroughly" overcorrects - the subagent loses the quality bar entirely and results become unpredictable.

Question 58 (Set B)

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

- (A) Update the synthesis agent to render each content type appropriately-financial data as tables, news as prose
- (B) Standardize all subagent outputs to prose summaries with inline citations
- (C) Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation
- (D) Standardize all subagent outputs to JSON with fields for 'claim', 'evidence', 'source', and 'confidence'

Answer: (A) Update the synthesis agent to render each content type appropriately-financial data as tables, news as prose

Confidence: high | basis: source-key - identical to 58 (Set A) / RAW-EH-009, answer confirmed

EXPLANATION

Each source has a natural shape, and flattening everything to bullets destroys it. Rendering each content type appropriately - financial figures as tables, news as prose - preserves the comparability of the numbers and the narrative flow of the reporting, which is exactly what the briefings lost.

Why the others are wrong

- **B.** Standardising on prose destroys tabular comparability for the financial data, trading one loss for another.
- **C.** A common intermediate representation is a conversion step that still has to be rendered at the end, and it tends to converge on generic formatting - the original problem.
- **D.** A uniform claim/evidence/source/confidence JSON improves structure for the pipeline but does not improve the human-readable briefing, which is what the question asks about.

Question 59 (Set B)

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

- (A) Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.
- (B) Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.
- (C) Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.
- (D) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.

Answer: (D) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.

Confidence: high | basis: same question as 59 (Set A) with options reordered - RAW-EH-005

EXPLANATION

The coordinator owns iterative refinement. Evaluating the synthesis for gaps and re-delegating targeted follow-up queries closes the specific holes before the report is written, without discarding the work already done - a feedback loop rather than a one-way pipeline.

Why the others are wrong

- **A.** Broadening the initial queries is a blunt, upfront guess that costs more on every run and still cannot guarantee it covers gaps only discovered after synthesis.
- **B.** Giving the synthesis agent its own web search tools breaks separation of concerns and bypasses the coordinator's control over delegation.
- **C.** Noting the unanswered questions improves transparency but leaves the report incomplete; the stated goal is completeness.

Question 60 (Set B)

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- (A) Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations
- (B) Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis
- (C) Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents
- (D) Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity

Answer: (B) Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis

Confidence: high | basis: source-key - identical to 50 / RAW-EH-006, answer confirmed

EXPLANATION

The precedents are independent, so they can be analysed concurrently. Having the coordinator spawn parallel subagents over subsets and aggregate their results cuts latency while keeping orchestration centralised - the coordinator still sees every delegation and every result, which is what preserves monitoring and debugging.

Why the others are wrong

- **A.** Letting the subagent spawn its own subagents dynamically decentralises orchestration; the coordinator loses visibility into agents it never created.
- **C.** A worker-pool message queue improves throughput but adds infrastructure and moves execution out of the coordinator's view, reducing debuggability.
- **D.** A recursive hierarchy compounds that problem - execution paths become deep and hard to trace, for no gain over one level of fan-out.

